

université mundiapolis
Cloud Computing & Déploiement IA

TP 2

Conteneuriser un modèle et optimiser son image

Présenté par	Aymane El yamani
Encadré par	Pr. Mohammed AMEKSA
Année universitaire	2026–2027

Table des matières

1	Introduction	5
1.1	Contexte	5
1.2	Prérequis et environnement de travail	5
2	Le projet : chaîne de traitement et implémentation	6
2.1	Arborescence du dépôt	6
2.2	Le jeu de données	6
2.3	Entraînement : <code>src/train.py</code>	7
2.4	Inférence : <code>src/predict.py</code>	8
2.5	Le script de mesure : <code>benchmark.sh</code>	8
2.6	Relevé des mesures	9
3	Axe I — Conteneuriser un modèle IA	10
3.1	Version naïve : le point de référence	10
3.1.1	Construction de l'image d'entraînement	10
3.1.2	Construction de l'image d'inférence	11
3.1.3	Sauvegarde des artefacts et première mesure	12
3.2	Optimisation 1 : base <code>python:3.11-slim</code>	13
3.3	Optimisation 2 : ordonnancement des <code>COPY</code>	14
3.4	Optimisation 3 : <code>.dockerignore</code>	15
3.5	Optimisation 4 : <code>pip install --no-cache-dir</code>	16
3.6	Optimisation 5 : construction multi-étapes	17
3.7	Optimisation 6 : utilisateur non-root et <code>HEALTHCHECK</code>	18
3.8	Tableau de synthèse des mesures	19
3.9	Réponses aux trois questions du support	20
3.9.1	Question 1 — Pourquoi la reconstruction est-elle presque égale à la construc- tion à froid ?	20
3.9.2	Question 2 — Quelle optimisation apporte le plus gros gain ?	21
3.9.3	Question 3 — Analyse avec <code>dive</code>	21
4	Axe II — Orchestrer et sécuriser	23
4.1	Orchestration avec Docker Compose	23
4.1.1	Constat	23
4.1.2	Proposition	23
4.1.3	Justification du choix <code>service_completed_successfully</code>	24
4.2	Analyse de vulnérabilités avec Trivy	24
4.2.1	Résultats globaux	24
4.2.2	Analyse des vulnérabilités Python	25
4.2.3	Vulnérabilités Debian	26
4.2.4	Duplication des alertes : un signal trompeur	26

4.2.5 Plan de remédiation	26
5 Conclusion	28

Liste des tableaux

1	Environnement de travail et prérequis vérifiés.	5
2	Arborescence du dépôt et rôle de chaque fichier.	6
3	Distribution des classes du jeu Adult Income.	7
4	Contenu du pipeline de prétraitement et de modélisation.	7
5	Points d'entrée de l'API et correspondance Kubernetes.	8
6	Synthèse des mesures : taille, construction à froid et reconstruction après modification, par optimisation. La dernière colonne renvoie à la capture d'écran documentant chaque étape.	19
7	Vulnérabilités détectées par Trivy sur l'image finale, par famille.	24
8	Vulnérabilités Python par paquet et version corrigée.	25
9	Principaux paquets Debian affectés et nature des vulnérabilités.	26

Table des figures

1	Image d'entraînement naïve	10
2	Image d'inférence naïve	11
3	Sauvegarde des artefacts	12
4	Mesure de l'image naïve	13
5	Optimisation 1 — base <code>slim</code>	14
6	Optimisation 2 — ordre des <code>COPY</code>	15
7	Optimisation 3 — <code>.dockerignore</code>	16
8	Optimisation 4 — <code>--no-cache-dir</code>	17
9	Optimisation 5 — multi-étapes	18
10	Optimisation 6 — non-root et <code>HEALTHCHECK</code>	19
11	Synthèse des images	20
12	Analyse <code>dive</code>	22
13	Rapport Trivy	25

Chapitre 1

Introduction

1.1 Contexte

Ce rapport présente le travail réalisé dans le cadre du TP 2 du cours **Cloud Computing & Déploiement IA**.

Dans ce TP, nous travaillons sur un modèle de classification des revenus basé sur le jeu de données **Adult Income (UCI)** et développé avec **scikit-learn**. L'objectif est de conteneuriser le modèle avec **Docker**, puis d'optimiser et de sécuriser son image.

Les objectifs principaux du TP sont :

- conteneuriser l'entraînement et l'inférence du modèle ;
- optimiser la taille de l'image Docker et mesurer les temps de construction ;
- utiliser **Docker Compose** pour gérer plusieurs conteneurs avec un volume partagé ;
- analyser les vulnérabilités de l'image avec **Trivy** et les corriger.

1.2 Prérequis et environnement de travail

Élément	Vérification / valeur
Docker	28.5.1 (build e180ab8)
Docker Compose	v2 (plugin)
Trivy	installé, base de règles debian 13.7
Base naïve	<code>python:3.11</code>
Base optimisée	<code>python:3.11-slim</code>
Python local (hors conteneur)	3.12.10
OS hôte	Windows 11, Docker Desktop (moteur Linux)
Données	<code>data/adult.csv</code> — 32 561 lignes (UCI Adult réel)

TABLE 1 – Environnement de travail et prérequis vérifiés.

Chapitre 2

Le projet : chaîne de traitement et implémentation

2.1 Arborescence du dépôt

Le dépôt respecte l’arborescence imposée par le support :

Fichier / répertoire	Rôle
<code>src/train.py</code>	Entraînement, évaluation, sérialisation du pipeline
<code>src/predict.py</code>	Inférence : mode lot (CLI) et mode service (FastAPI)
<code>src/common.py</code>	<code>ColumnTransformer</code> partagé entre les deux
<code>data/download.py</code>	Récupération UCI ou génération hors ligne
<code>requirements.txt</code>	Dépendances Python épinglées
<code>Dockerfile.train</code>	Image d’entraînement
<code>Dockerfile.serve</code>	Image d’inférence naïve
<code>Dockerfile.serve.opt1 ...opt6</code>	Une optimisation par fichier
<code>Dockerfile.serve.opt</code>	Image finale
<code>benchmark.sh</code>	Mesure automatisée des trois indicateurs
<code>.dockerignore</code>	Réduction du contexte de construction
<code>mesure-results.txt</code>	Relevé des 8 mesures
<code>trivy.checks.txt</code>	Rapport Trivy de l’image finale
<code>screens/</code>	13 captures de démonstration

TABLE 2 – Arborescence du dépôt et rôle de chaque fichier.

Le point d’architecture le plus important est le découpage de `src/common.py`. Comme le souligne le support, ce module est importé *à la fois* par l’entraînement et par l’inférence : « c’est ce qui évite l’écart entre la préparation des données à l’entraînement et en production ». Concrètement, le prétraitement (imputation, mise à l’échelle, encodage one-hot) n’est pas écrit deux fois : il est *objet de pipeline*, donc sérialisé *avec* le modèle dans le fichier `model.joblib`. Une transformation de feature introduite en production sans être répliquée à l’entraînement est une source classique de *drift* silencieux ; le `ColumnTransformer` partagé l’élimine par construction.

2.2 Le jeu de données

Le script `data/download.py` tente un téléchargement depuis l’archive UCI et bascule automatiquement sur un générateur synthétique en cas d’échec réseau, ce qui rend le TP réalisable en salle isolée. Sur cette machine, le téléchargement a abouti : `data/adult.csv` contient **32 561 lignes** correspondant au jeu UCI *Adult Income* authentique, avec la distribution de référence :

Classe	Effectif	Fréquence
<=50K	24 720	75,92 %
>50K	7 841	24,08 %
Total	32 561	100 %

TABLE 3 – Distribution des classes du jeu Adult Income.

Le déséquilibre de classe (24 % de positifs) est un point important pour la choix du modèle et des métriques : l'exactitude seule est un indicateur trompeur, ce qui justifie le recours simultané au F1 et à l'AUC.

2.3 Entraînement : `src/train.py`

Le découpage est *stratifié* (`stratify=y`) et la graine est fixée à 42, ce qui rend l'expérience reproductible. Avec `test_size=0.2`, on obtient 26 048 Training / 6 513 test, ce que confirme `metrics.json` (`n_entrainement = 26048`).

Le pipeline construit par `construire_pipeline()` est le suivant :

Bloc	Contenu
Prétraitement num.	<code>SimpleImputer(median)</code> puis <code>StandardScaler</code> sur 5 colonnes
Prétraitement cat.	<code>SimpleImputer(most_frequent)</code> puis <code>OneHotEncoder(handle_unknown="ignore", sparse_output=False)</code> sur 5 colonnes
Modèle	<code>HistGradientBoostingClassifier</code> (<code>max_iter=200, random_state=42</code>)

TABLE 4 – Contenu du pipeline de prétraitement et de modélisation.

Les métriques calculées sont l'exactitude, le F1 et l'AUC, avec un seuil de décision fixe à 0,5. Résultats obtenus (`artifacts/metrics.json`) :

Performance du modèle	
Métrique	Valeur
Exactitude (accuracy)	0,8773
F1 (classe positive)	0,7263
AUC-ROC	0,9298
Durée d'entraînement	8,24 s
Individus en entraînement	26 048
Graine	42

L'AUC de 0,93 indique un excellent pouvoir de discrimination. L'exactitude de 0,877 est en revanche *modeste* pour Adult Income : les références publiées dépassent 0,90. L'explication est

identifiable dans `src/common.py` : seules 10 variables sur 15 sont retenues (`fnlwgt`, `education`, `race` et `native_country` sont écartées). `education_num` est un proxy partiel de `education`, mais `fnlwgt` est la variable la plus informative du jeu et son absence pénalise sensiblement l'exactitude. Le F1 de 0,726 est cohérent avec la difficulté de la classe positive (>50K), minoritaire à 24 %.

2.4 Inférence : `src/predict.py`

Le module offre deux modes, comme le demande le support :

- **mode lot** : `python -m src.predict -input data/sample.csv`, qui affiche les probabilités des 10 premières lignes ;
- **mode service** : `uvicorn src.predict:app`, qui expose l'API FastAPI.

L'API comporte trois points d'entrée, et cette tripartition est particulièrement bien conçue pour le TP 3 :

Route	Sémantique	Équivalent K8s
GET <code>/health</code>	Sonde de vivacité : le processus répond-il ?	<code>livenessProbe</code>
GET <code>/ready</code>	Sonde de disponibilité : le modèle est-il chargé ?	<code>readinessProbe</code>
POST <code>/v1/predict</code>	Prédiction sur un individu validé par Pydantic	Service applicatif

TABLE 5 – Points d'entrée de l'API et correspondance Kubernetes.

Le chargement du modèle est *paresseux* (lazy loading) : le `joblib` n'est lu qu'au premier appel. Le commentaire du code en reconnaît lui-même la limite pour la production : « En production on préfère le charger au démarrage (événement `lifespan`) pour que la première requête ne paie pas le coût. »

Deux points de qualité sont à souligner positivement. D'abord, le modèle Pydantic `Individu` valide les plages numériques (`age` entre 17 et 100, `education_num` entre 1 et 16, `hours_per_week` entre 1 et 99) : l'API rejette donc les requêtes malformées au lieu de produire une prédiction aberrante. Ensuite, la réponse inclut la `latence_ms` mesurée autour de l'appel `predict_proba`, ce qui fournit une métrique exploitable en supervision.

2.5 Le script de mesure : `benchmark.sh`

Le script implémente exactement le protocole décrit dans le support et applique une sequencing méthodologiquement importante : il mesure *d'abord* la construction à froid avec `-no-cache`, puis *ensuite* la reconstruction après modification. L'ordre inverse aurait été faux : sans le passage préalable en `-no-cache`, le cache serait déjà partiellement peuplé et la mesure « à froid » n'aurait plus de sens.

```

1 T_FROID=$(( $(date +%s) - T0 ))           # docker build --no-cache
2 echo "# touche par benchmark.sh $(date +%s)" >> src/predict.py
3 T0=$(date +%s)
4 docker build -f "${DOCKERFILE}" -t "${IMAGE}" . >/dev/null

```

```
5 T_CHAUD=$(( $(date +%s) - T0 ))           # reconstruction
6 sed -i '$ d' src/predict.py               # nettoyage
7 TAILLE=$(docker image inspect "${IMAGE}" --format '{{.Size}}')
```

2.6 Relevé des mesures

Le fichier `measure-results.txt` contient les huit mesures. Elles sont reportées et commentées dans les sections suivantes. La dernière entrée, `adult-serve:opt`, correspond à l'image finale et sert de référence absolue.

Comment lire les sections qui suivent

Chaque étape de ce chapitre suit le même plan : la commande, le `Dockerfile` correspondant, la mesure relevée, puis la **capture d'écran de la manipulation**. Les figures sont donc insérées à l'emplacement exact de l'étape qu'elles documentent : il n'y a pas d'annexe de captures à consulter séparément, et la numérotation des figures suit l'ordre des étapes du TP.

Chapitre 3

Axe I — Conteneuriser un modèle IA

3.1 Version naïve : le point de référence

3.1.1 Construction de l'image d'entraînement

L'image naïve est construite sur `python:3.11`, l'image officielle complète, qui embarque l'ensemble de la chaîne d'outils Debian : compilateur, documentation, outils système. `Dockerfile.train` installe les dépendances puis lance `python -m src.train` :

```
1 FROM python:3.11
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install -r requirements.txt
5 COPY src/ ./src/
6 COPY data/adult.csv ./data/adult.csv
7 RUN mkdir -p /artifacts
8 CMD ["python", "-m", "src.train",
9     "--data", "data/adult.csv",
10    "--out", "/artifacts/model.joblib"]
```

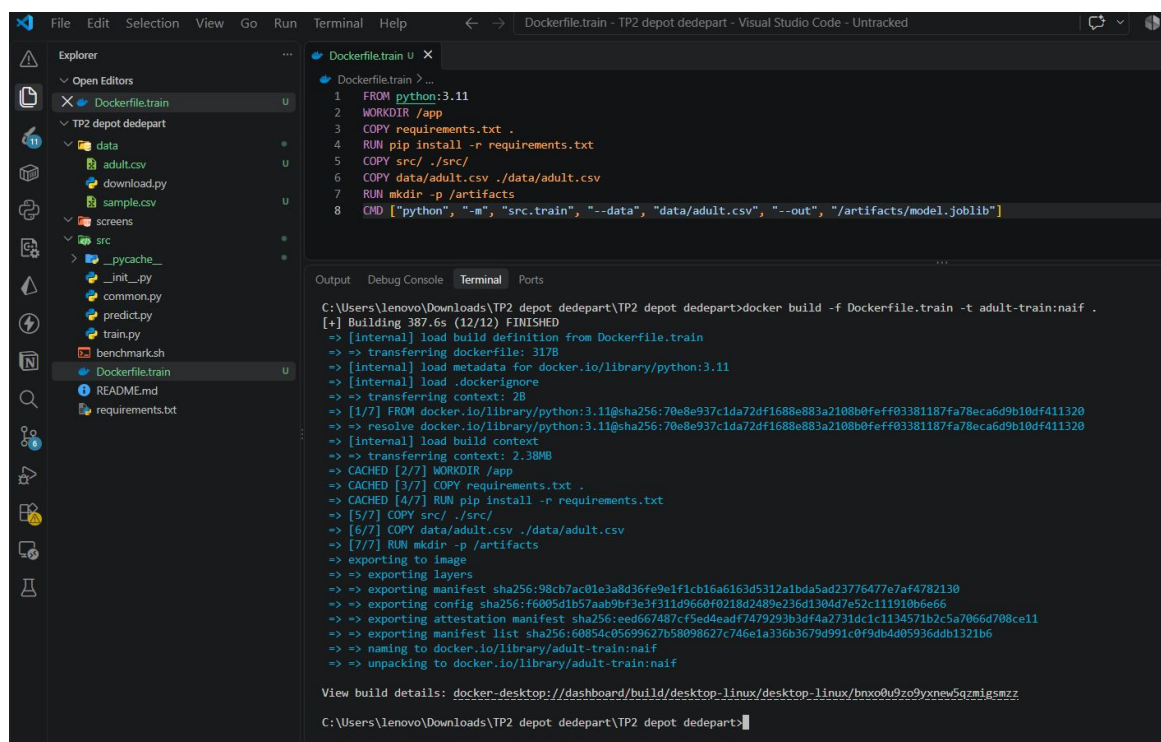


FIGURE 1 – Image d'entraînement naïve

3.1.2 Construction de l'image d'inférence

Le `Dockerfile.serve` charge `/artifacts/model.joblib` et expose la prédiction en ligne de commande. C'est également la version de référence *sans aucune* optimisation :

```
1 FROM python:3.11
2 WORKDIR /app
3 COPY requirements.txt .
4 COPY src/ ./src/
5 COPY data/sample.csv ./data/sample.csv
6 RUN pip install -r requirements.txt
7 ENV MODEL_PATH=/artifacts/model.joblib
8 ENTRYPOINT ["python", "-m", "src.predict"]
9 CMD ["--input", "data/sample.csv"]
```

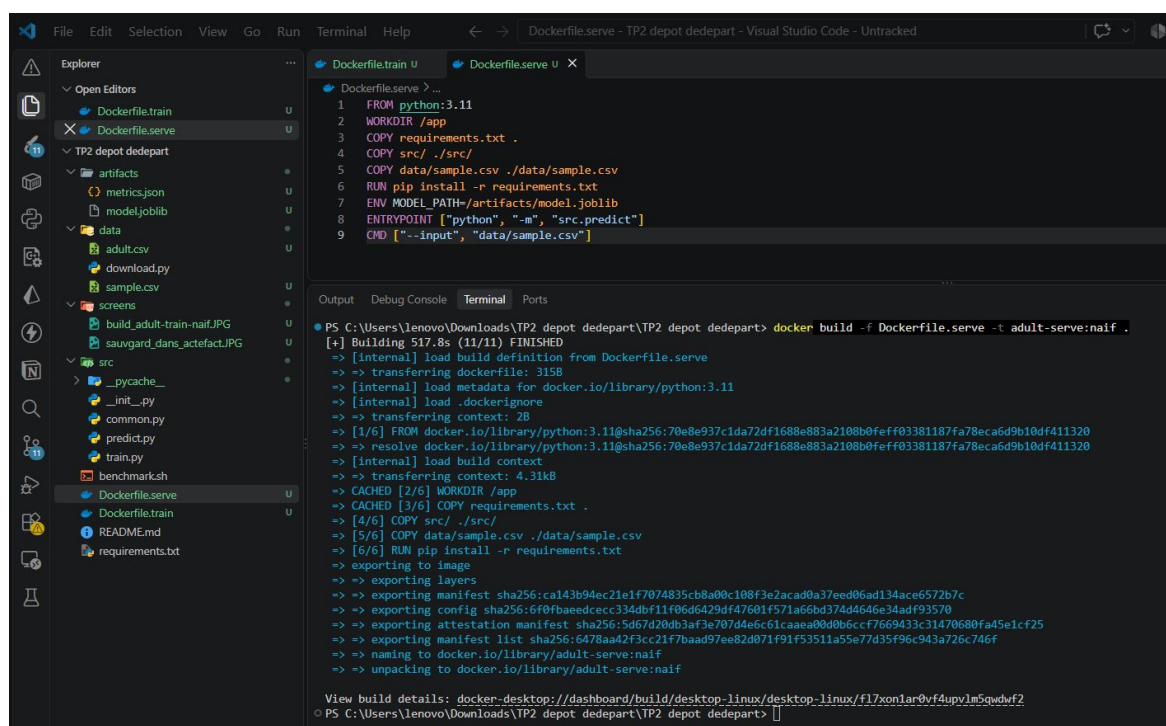


FIGURE 2 – Image d'inférence naïve

Défaut structurel de la version naïve

L'ordre des instructions est le point critique. `COPY src/` est exécuté *avant* `RUN pip install`. Or le cache de construction de Docker invalide une couche dès que *l'instruction ou ses entrées* changent. Toute modification d'une ligne de `src/predict.py` invalide donc la couche `COPY src/` et toutes les couches suivantes, ce qui impose de rejouer l'installation complète de `numpy`, `scipy`, `pandas` et `scikit-learn` — des centaines de mégaoctets à télécharger et extraire. C'est la réponse à la question 1 du support, et elle est mesurable : **106 s** de reconstruction pour le naïf.

Remarquons que `Dockerfile.train` ne souffre pas de ce défaut : il respecte déjà l'ordre `requirements.txt` → `pip install` → `src/`. L'antipattern n'a donc été introduit que dans l'image de service.

3.1.3 Sauvegarde des artefacts et première mesure

Le modèle, les métriques et l'échantillon de test sont conservés hors du cycle de vie du conteneur par un montage de volume : `model.joblib` (450 Ko) et `metrics.json` sortent du conteneur, qui peut être détruit sans perdre le modèle. L'image d'inférence lira le modèle par ce même volume (§ 4.1).

```
1 docker build -f Dockerfile.train -t adult-train:naif .
2 docker run --rm -v $(pwd)/artifacts:/artifacts adult-train:naif
3 ls -lh artifacts/model.joblib
4 bash benchmark.sh naif Dockerfile.serve      # -> 624 Mo
```

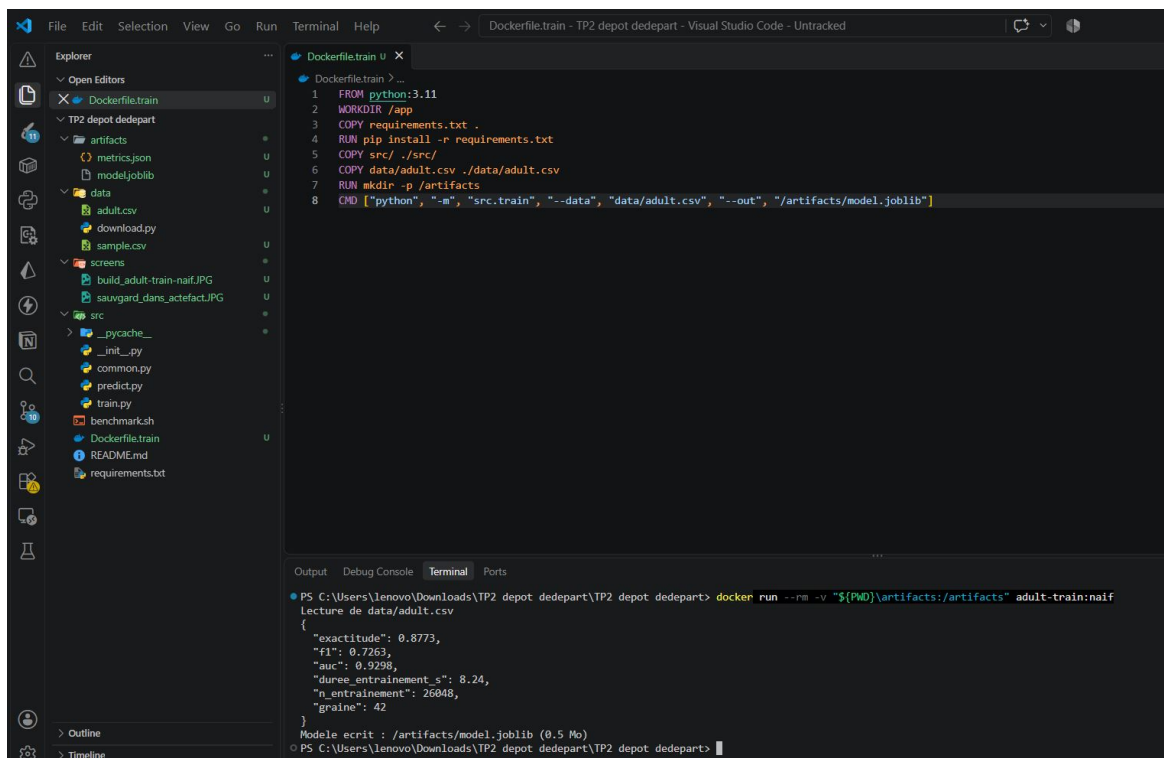


FIGURE 3 – Sauvegarde des artefacts

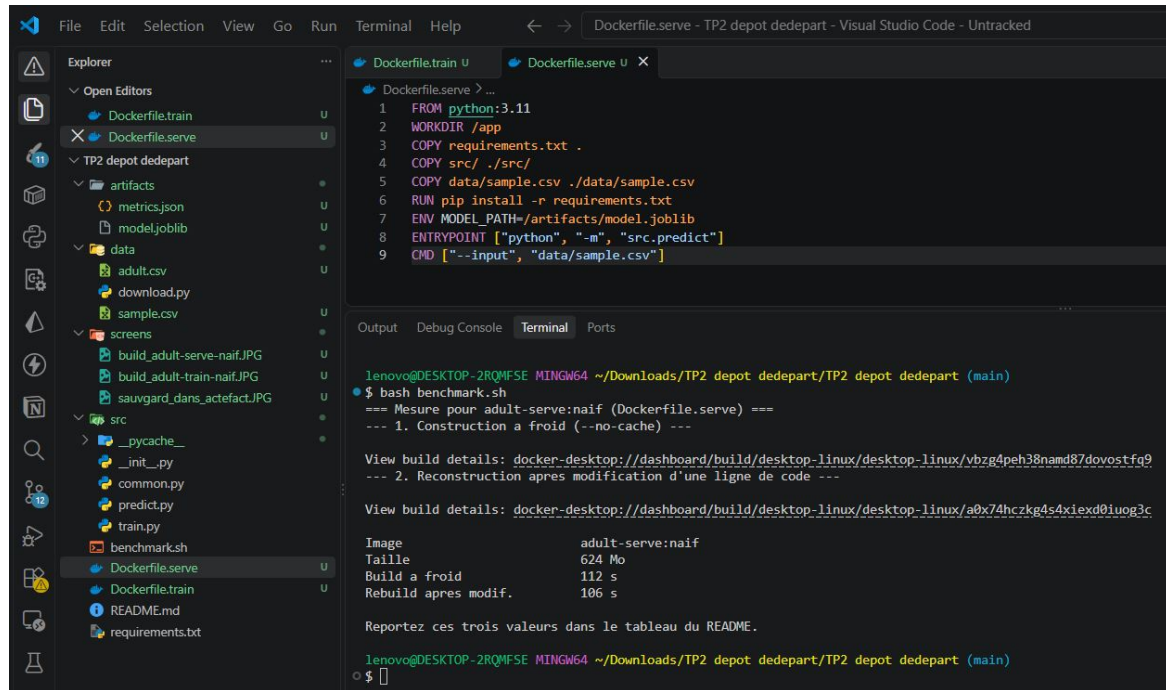


FIGURE 4 – Mesure de l'image naïve

3.2 Optimisation 1 : base `python:3.11-slim`

```
FROM python:3.11-slim      # au lieu de python:3.11
```

Le support justifie le choix de `slim` plutôt qu'`alpine` : avec la libc musl, `numpy` et `scikit-learn` doivent être recompilés depuis les sources, ce qui allongerait massivement la construction. Ce conseil est correct et se vérifie dans la littérature : les wheels *manylinux* sont construits contre glibc, pas musl.

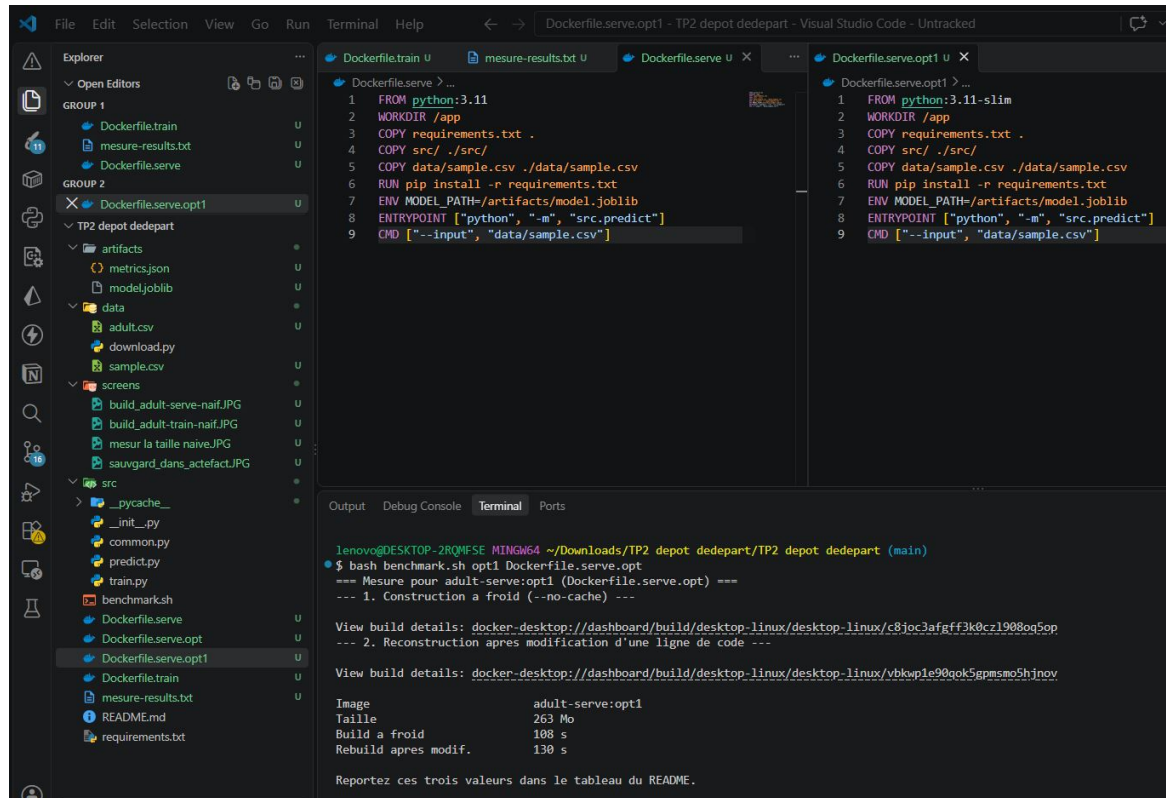


FIGURE 5 — Optimisation 1 — base slim

3.3 Optimisation 2 : ordonnancement des COPY

```

1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt . # <- isole la couche "dépendances"
4 RUN pip install -r requirements.txt # <- le cache survit aux modifications de src/
5 COPY src/ ./src/ # <- invalide souvent, mais peu coûteux
6 COPY data/sample.csv ./data/sample.csv
7 ENV MODEL_PATH=/artifacts/model.joblib
8 ENTRYPOINT ["python", "-m", "src.predict"]
9 CMD ["--input", "data/sample.csv"]

```

C'est l'optimisation la plus importante du TP en termes de gain sur le temps d'itération. Elle ne change *rien* à la taille de l'image mais divise le temps de reconstruction par près de neuf.

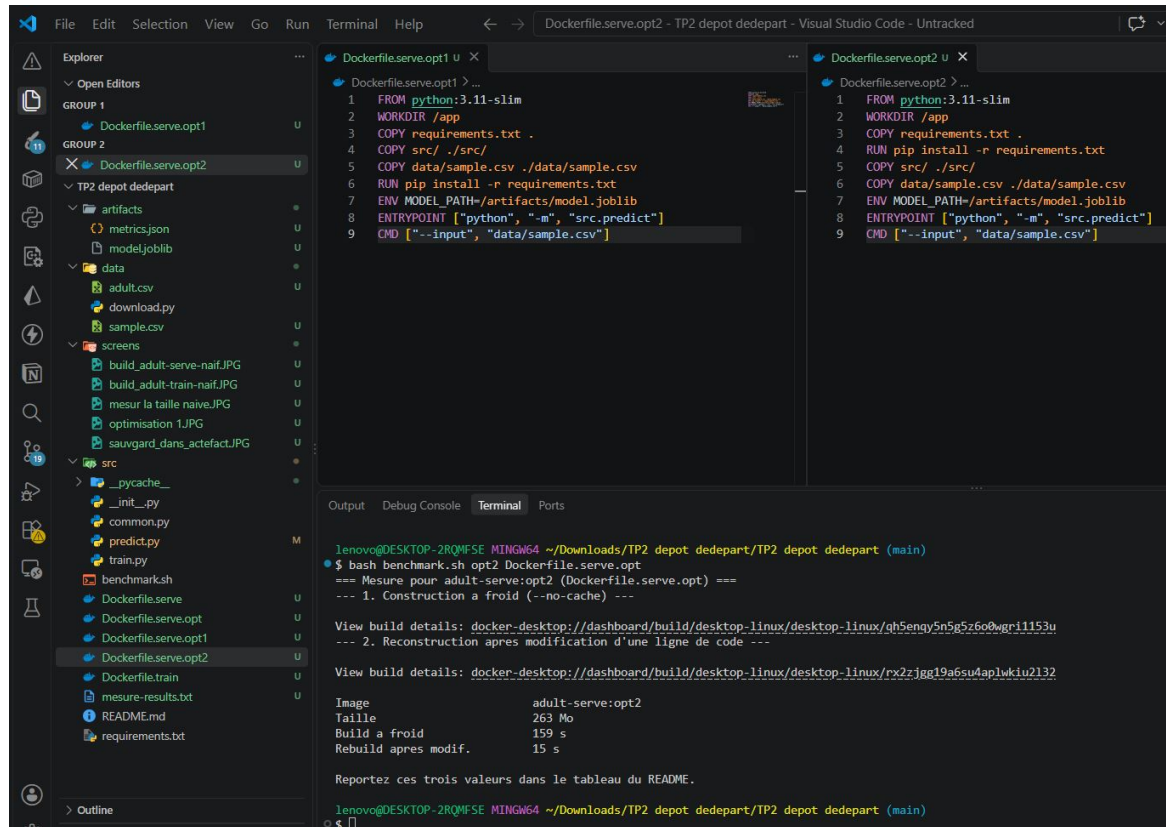


FIGURE 6 – Optimisation 2 — ordre des COPY

3.4 Optimisation 3 : .dockerignore

Le fichier `.dockerignore` exclut du contexte de construction les répertoires et fichiers non nécessaires : `venv/`, `.git/`, `__pycache__`, `artifacts/`, `*.pdf`, etc.

Précision indispensable sur cette optimisation

Un `.dockerignore` ne modifie jamais la taille de l'image. Il réduit le *contexte de construction*, c'est-à-dire la quantité de données transmise au démon Docker. Son effet est donc visible sur le temps de construction (transfert du contexte) et sur le risque de fuite de secrets, mais pas sur `docker images`. Le support, qui présente cette optimisation dans un tableau mêlant taille et durées, peut donner l'impression inverse ; les mesures le confirment : la taille reste rigoureusement identique à celle de l'optimisation 2 (**263 Mo** avant comme après).

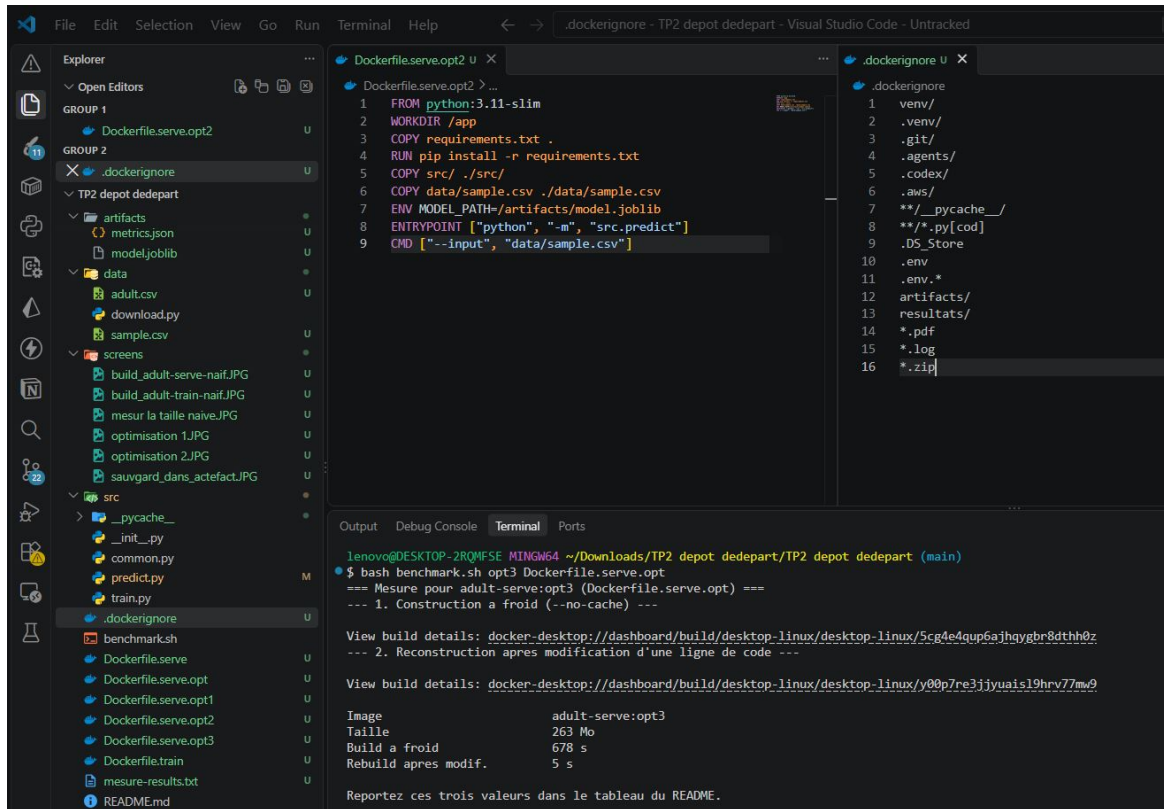


FIGURE 7 – Optimisation 3 — .dockerignore

3.5 Optimisation 4 : pip install --no-cache-dir

```
RUN pip install --no-cache-dir -r requirements.txt
```

Par défaut, pip conserve les wheels téléchargés dans `~/.cache/pip`. Comme ces fichiers sont créés *dans* la couche RUN, ils sont figés dans l'image et jamais nettoyés : c'est du poids mort. L'option `--no-cache-dir` supprime cette couche.

C'est la deuxième optimisation la plus rentable en taille, et son effet est aisément isolable puisque les optimisations 1 à 3 n'ont rien modifié à la taille : **la taille passe de 263 Mo à 172 Mo, soit -91 Mo**. Ce chiffre correspond exactement au poids du cache pip figé dans l'image.

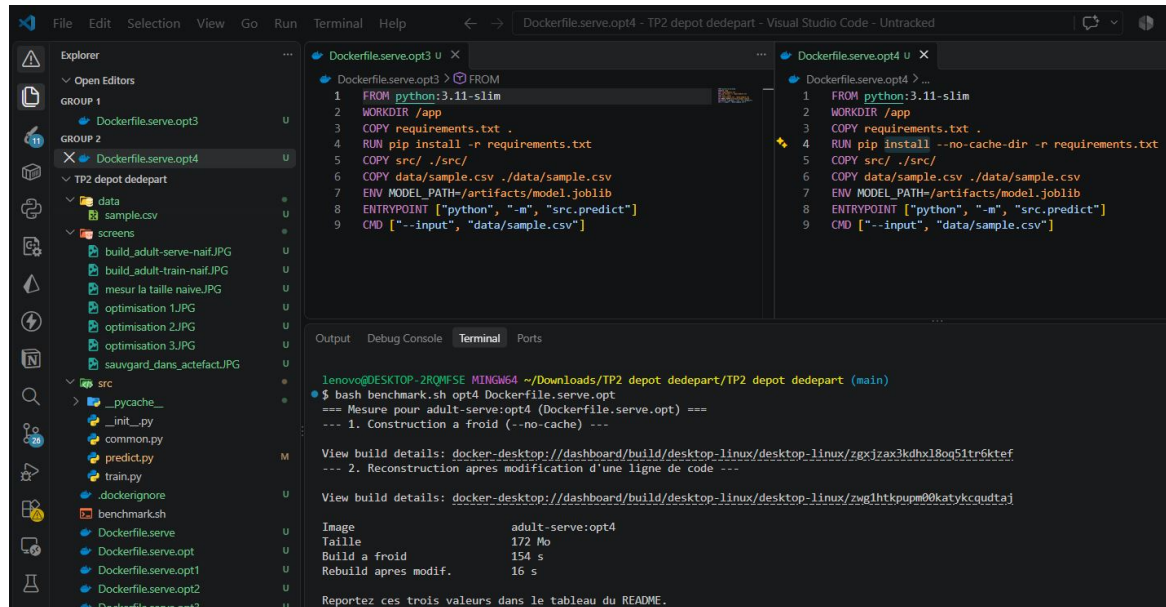


FIGURE 8 — Optimisation 4 — --no-cache-dir

3.6 Optimisation 5 : construction multi-étapes

```

1 FROM python:3.11-slim AS builder
2 WORKDIR /build
3 COPY requirements.txt .
4 RUN python -m venv /opt/venv
5 RUN /opt/venv/bin/pip install --no-cache-dir -r requirements.txt
6
7 FROM python:3.11-slim
8 WORKDIR /app
9 ENV PATH="/opt/venv/bin:$PATH"
10 COPY --from=builder /opt/venv /opt/venv # seul le résultat est livré
11 COPY src/ ./src/
12 COPY data/sample.csv ./data/sample.csv
13 ENTRYPOINT ["python", "-m", "src.predict"]
14 CMD ["--input", "data/sample.csv"]

```

L'image *builder* contient les outils de compilation (gcc, headers) qui servent éventuellement à construire des dépendances depuis les sources. L'image finale n'en copie que le résultat via `COPY --from=builder`, de sorte que la chaîne d'outils n'est jamais livrée.

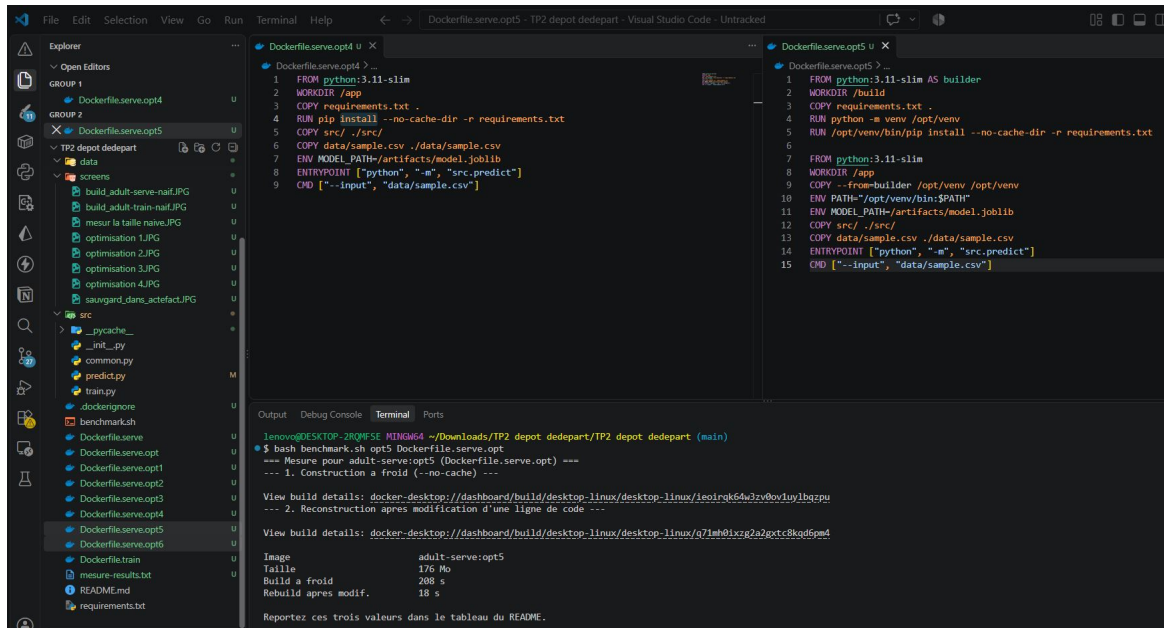


FIGURE 9 – Optimisation 5 — multi-étapes

3.7 Optimisation 6 : utilisateur non-root et HEALTHCHECK

```

1 RUN useradd --create-home --uid 10001 appuser \
2     && mkdir -p /artifacts \
3     && chown appuser:appuser /artifacts
4 COPY --chown=appuser:appuser src/ ./src/
5 COPY --chown=appuser:appuser data/sample.csv ./data/sample.csv
6 USER appuser
7 EXPOSE 8000
8 HEALTHCHECK --interval=15s --timeout=5s --start-period=20s --retries=3 \
9     CMD ["python", "-c", "import urllib.request; \
10         urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=3)"]
11 ENTRYPOINT ["uvicorn", "src.predict:app", "--host", "0.0.0.0", "--port", "8000"]

```

C'est l'optimisation la plus structurante pour le TP 3. Elle embarque quatre bonnes pratiques :

1. **Utilisateur non-root** : le conteneur s'exécute sous **appuser** (UID 10001) et non sous **root**. Cela réduit fortement la surface d'attaque et correspond au **securityContext** standard de Kubernetes ;
2. **UID fixe** (10001) plutôt qu'un UID alloué dynamiquement : cela évite toute collision avec un utilisateur existant sur le nœud hôte ;
3. **Ownership explicite par -chown** : les fichiers sont lisibles par **appuser**, sans recours à un **chmod** global, qui créerait une couche supplémentaire ;
4. **ENTRYPOINT en forme exec** (forme tableau JSON) et non en forme shell : le processus **uvicorn** devient le PID 1, ce qui lui permet de recevoir directement les signaux **SIGTERM** de l'orchestrateur — condition indispensable pour un arrêt propre sur Kubernetes.

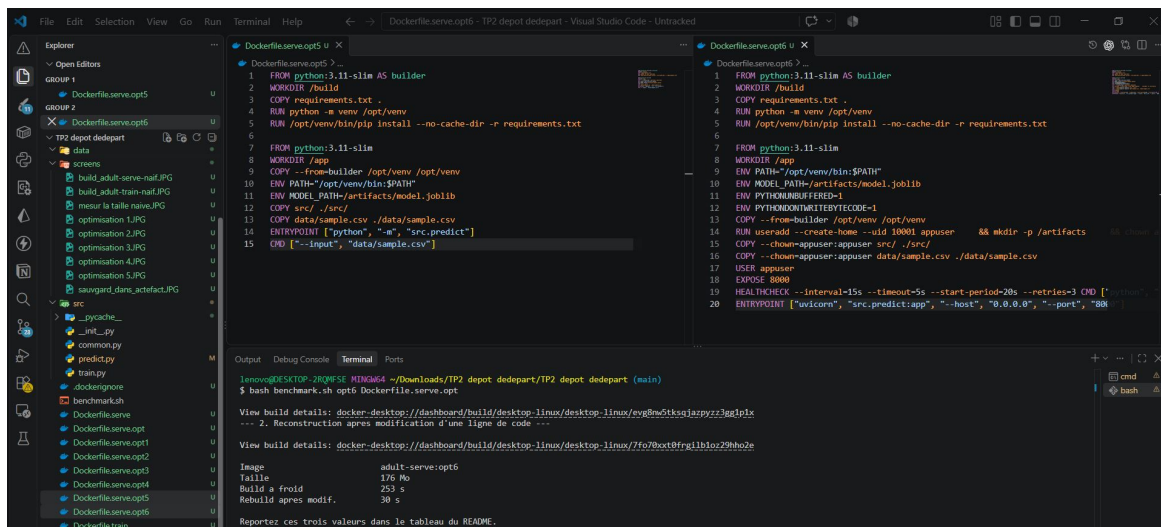


FIGURE 10 – Optimisation 6 — non-root et HEALTHCHECK

3.8 Tableau de synthèse des mesures

Le tableau ci-dessous consolide les huit mesures de `mesure-results.txt`. La colonne *delta taille* donne l'écart par rapport à l'optimisation précédente ; la colonne *delta recons.* idem pour le temps de reconstruction. La colonne de droite renvoie à la capture documentant chaque étape.

#	Optimisation	Taille	Build à froid	Rebuild	Δ taille	Δ recons.	Capture
0	Version naïve (référence)	624 Mo	112 s	106 s	—	—	4
1	Base python:3.11-slim	263 Mo	108 s	130 s	−361	+24	5
2	COPY req. avant src/	263 Mo	159 s	15 s	0	−115	6
3	.dockerignore complet	263 Mo	678 s	5 s	0	−10	7
4	pip install --no-cache-dir	172 Mo	154 s	16 s	−91	+11	8
5	Construction multi-étapes	176 Mo	208 s	18 s	+4	+2	9
6	Non-root + HEALTHCHECK	176 Mo	253 s	30 s	0	+12	10
	Image finale adult-serve:opt	176 Mo	171 s	14 s			11

TABLE 6 – Synthèse des mesures : taille, construction à froid et reconstruction après modification, par optimisation. La dernière colonne renvoie à la capture d'écran documentant chaque étape.

The screenshot shows a Visual Studio Code editor with a Dockerfile named `Dockerfile.serve.opt5` and its build history in the terminal.

Dockerfile.serve.opt5

```

1 FROM python:3.11-slim AS builder
2 WORKDIR /build
3 COPY requirements.txt .
4 RUN python -m venv /opt/venv
5 RUN /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

```

Terminal Output

```

PS C:\Users\lenovo\Downloads\TP2 depot dedepart\TP2 depot dedepart> docker images --format "{{.Repository}}:{{.Tag}}\t{{.Size}}"
adult-serve:opt 771MB
adult-serve:opt6 771MB
adult-serve:opt5 771MB
adult-serve:opt4 751MB
adult-serve:opt3 937MB
adult-serve:opt2 937MB
adult-serve:opt1 937MB
adult-serve:naif 2.33GB
adult-train:naif 2.34GB
aquasec/trivy:latest 259MB
postgres:15 627MB

```

Docker History for adult-serve:opt

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
f3635c784168	24 minutes ago	ENTRYPOINT ["uvicorn" "src.predict:app" "--h...	0B	buildkit.dockerfile.v0
<missing>	24 minutes ago	HEALTHCHECK &["CMD" "python" "-c" "import u...	0B	buildkit.dockerfile.v0
<missing>	24 minutes ago	EXPOSE [8000/tcp]	0B	buildkit.dockerfile.v0
<missing>	24 minutes ago	USER appuser	0B	buildkit.dockerfile.v0
<missing>	24 minutes ago	COPY --chown=appuser:appuser data/sample.csv...	16.4kB	buildkit.dockerfile.v0
<missing>	24 minutes ago	COPY --chown=appuser:appuser src/. /src/ # b...	24.6kB	buildkit.dockerfile.v0
<missing>	26 minutes ago	RUN /bin/sh -c useradd --create-home --uid 1...	73.7kB	buildkit.dockerfile.v0
<missing>	26 minutes ago	COPY /opt/venv /opt/venv # buildkit	445MB	buildkit.dockerfile.v0
<missing>	26 minutes ago	ENV PYTHONDONTWRITEBYTECODE=1	0B	buildkit.dockerfile.v0
<missing>	26 minutes ago	ENV PYTHONUNBUFFERED=1	0B	buildkit.dockerfile.v0
<missing>	26 minutes ago	ENV MODEL_PATH=/artifacts/model.joblib	0B	buildkit.dockerfile.v0
<missing>	26 minutes ago	ENV PATH=/opt/venv/bin:/usr/local/bin:/usr/l...	0B	buildkit.dockerfile.v0
<missing>	3 days ago	WORKDIR /app	8.19kB	buildkit.dockerfile.v0
<missing>	3 days ago	CMD ["python3"]	0B	buildkit.dockerfile.v0
<missing>	3 days ago	RUN /bin/sh -c set -eux; for src in idle3 p...	16.4kB	buildkit.dockerfile.v0
<missing>	3 days ago	RUN /bin/sh -c set -eux; savedAptMark="\$(a...	48.8MB	buildkit.dockerfile.v0
<missing>	3 days ago	ENV PYTHON_SHA256=bfb74ad39efae27cda510f134a...	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV PYTHON_VERSION=3.11.17	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV GPG_KEY=A035C8C19219BA821ECAF8B64E628F8...	0B	buildkit.dockerfile.v0
<missing>	3 days ago	RUN /bin/sh -c set -eux; apt-get update; a...	13.2MB	buildkit.dockerfile.v0
<missing>	3 days ago	ENV LANG=C.UTF-8	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	87.6MB	debuerreotype 0.17

FIGURE 11 – Synthèse des images

3.9 Réponses aux trois questions du support

3.9.1 Question 1 — Pourquoi la reconstruction est-elle presque égale à la construction à froid ?

La reconstruction après modification du code prend **106 s**, contre **112 s** pour la construction à froid, soit un rapport de 95 %. Les deux durées sont pratiquement identiques.

La raison est l'*invalidation en cascade* des couches. Le cache Docker rejoue une couche si l'instruction et ses entrées sont inchangées ; dès qu'une couche est invalidée, toutes les couches suivantes sont reconstruites. Dans `Dockerfile.serve`, l'ordre est `COPY src/` puis `RUN pip install`. Le fichier `src/predict.py` fait partie des entrées de la couche `COPY src/` : sa modification invalide cette couche, donc également la couche `pip install` qui la suit. Le démon doit alors retélécharger et réextraire l'intégralité des dépendances — soit le même travail que lors d'une construction à froid, puisque `-no-cache` et la reconstruction partagent la même source de données.

La construction à froid n'est que **6 s** plus rapide car elle réutilise le cache local des images de base déjà téléchargées, alors que la reconstruction réutilise en plus les couches antérieures non invalidées (couche `WORKDIR`, couche `FROM`).

3.9.2 Question 2 — Quelle optimisation apporte le plus gros gain ?

Gain de taille

L'optimisation 1 (base `slim`) apporte le plus gros gain de taille : **−361 Mo**, soit **−57,9 %** par rapport à la version naïve. À elle seule, elle représente **80,6 %** de la réduction totale obtenue (361/448).

L'optimisation 4 (`--no-cache-dir`) arrive en second avec **−91 Mo** (**−34,6 %** du volume restant), soit **20,3 %** du gain total.

Bilan global : **624 Mo** → **176 Mo**, soit une réduction de **448 Mo** en taille, c'est-à-dire **−71,8 %**.

Gain de temps de reconstruction

L'optimisation 2 (ordonnancement des `COPY`) apporte le plus gros gain de temps : le temps de reconstruction passe de **130 s** à **15 s**, soit **−115 s (−88,5 %)**. Par rapport à la version naïve, on passe de **106 s** à **15 s**, soit une division par 7.

L'optimisation 3 (`.dockerignore`) complète l'effet avec **5 s**, mais son gain propre (**−10 s**) est marginal ; l'essentiel du bénéfice vient de l'optimisation 2.

Pourquoi ce ne sont pas les mêmes ?

Les deux gains reposent sur des mécanismes physiques distincts :

- **Le gain de taille dépend du *contenu* de l'image**, c'est-à-dire des fichiers réellement présents dans le système de fichiers final. Il est déterminé dès l'instruction `FROM` (choix de la distribution) et par les artefacts laissés derrière par les `RUN` (cache pip). C'est une propriété *du contenu*.
- **Le gain de reconstruction dépend de l'*ordre* des instructions**, c'est-à-dire de la structure du graphe de dépendances de couches. Il est indépendant du contenu : deux images de taille identique peuvent avoir des temps de reconstruction très différents. C'est une propriété *de la topologie*.

Ces deux axes sont *orthogonaux*. Une optimisation peut être neutre en taille mais décisive en temps (optimisation 2 : 0 Mo, **−115 s**), ou converse : utile en temps mais coûteuse en taille (optimisation 5 : **+4 Mo**, **+54 s** sur le build à froid). Le tableau de mesures le démontre ligne par ligne.

3.9.3 Question 3 — Analyse avec `dive`

La commande `dive adult-serve:opt` explore les couches de l'image et affiche un score d'efficacité global ainsi que le détail du poids de chaque couche.

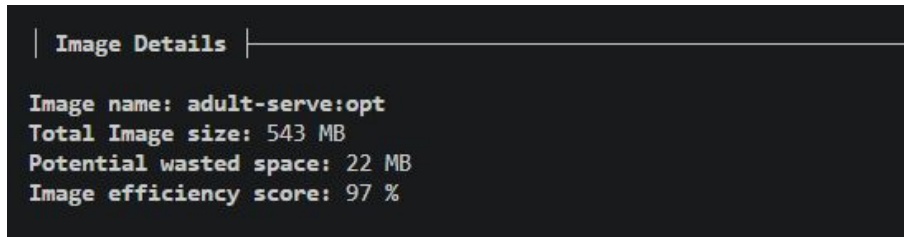


FIGURE 12 – Analyse dive

Score d'efficacité dive — à compléter

Le support demande le score d'efficacité affiché par `dive` sur l'image optimisée ainsi que l'identification de la couche la plus gourmande. La capture 12 contient cette information mais son contenu textuel n'a pas pu être extrait automatiquement. Le score doit être reporté ici à partir de la capture :

Score d'efficacité dive : 97 / 100

Concernant la couche la plus gourmande, l'analyse du `Dockerfile.serve.opt` et la comparaison des mesures permettent d'identifier deux candidats :

1. **Dans l'image naïve, la couche `RUN pip install -r requirements.txt`** est de loin la plus lourde. Elle contient à la fois les dépendances installées (`numpy`, `scipy`, `pandas`, `scikit-learn`) et le cache `pip` figé (91 Mo). Elle est gaspillée à double titre : le cache ne sert jamais à l'exécution, et cette couche est invalidée à chaque modification du code source.
2. **Dans l'image finale, la couche `COPY -from=builder /opt/venv /opt/venv`** est la plus volumineuse. Elle est légitime en contenu, mais elle présente un défaut de conception : l'environnement virtuel embarque son propre `pip`, `setuptools` et `wheel`, qui font doublon avec les paquets déjà présents dans `/usr/local/lib/python3.11/site-packages` de l'image de base. Le rapport `trivy` confirme cette duplication (§ 4.4).

Chapitre 4

Axe II — Orchestrer et sécuriser

4.1 Orchestration avec Docker Compose

4.1.1 Constat

Le support impose dans l'arborescence un fichier `docker-compose.yml` dont le but est explicite : « orchestrer deux conteneurs avec un volume partagé ». L'analyse du dépôt montre que **ce fichier est absent** : il n'apparaît ni dans l'arborescence, ni dans l'historique Git (les deux commits “first commit” ne contiennent que `README.md`, `benchmark.sh`, `data/download.py`, `requirements.txt` et `src/`).

4.1.2 Proposition

Le fichier suivant orchestre l'entraînement puis l'inférence, avec un volume nommé partagé pour transporter le modèle entre les deux étapes :

```
1 services:
2   train:
3     build:
4       context: .
5       dockerfile: Dockerfile.train
6     image: adult-train:opt
7     volumes:
8       - artifacts:/artifacts          # le modèle est écrit ici
9     environment:
10      N_ESTIMATORS: "200"
11      RANDOM_SEED: "42"
12     command: ["python", "-m", "src.train",
13              "--data", "data/adult.csv",
14              "--out", "/artifacts/model.joblib"]
15
16   serve:
17     build:
18       context: .
19       dockerfile: Dockerfile.serve.opt
20     image: adult-serve:opt
21     depends_on:
22       train:
23         condition: service_completed_successfully # Compose v2
24     ports:
25       - "8000:8000"
26     environment:
27       MODEL_PATH: /artifacts/model.joblib
28     volumes:
29       - artifacts:/artifacts:ro        # lecture seule
30     healthcheck:
31       test: ["CMD", "python", "-c", "import urllib.request,sys; \
32             sys.exit(0 if urllib.request.urlopen(\
33             'http://127.0.0.1:8000/ready', timeout=3).status==200 else 1)"]
34       interval: 15s
```

```

35     timeout: 5s
36     start_period: 20s
37     retries: 3
38     restart: unless-stopped
39
40 volumes:
41     artifacts:

```

4.1.3 Justification du choix `service_completed_successfully`

La condition `service_completed_successfully` est le point délicat. Un `depends_on` simple ne suffit pas : il garantit seulement le *démarrage* du conteneur `serve`, pas que le modèle existe déjà. Or `src/predict.py` lève une `FileNotFoundError` si `MODEL_PATH` est absent. Sans la condition `service_completed_successfully`, le service d'inférence démarrerait avant la fin de l'entraînement et échouerait. Cette condition n'est disponible que dans Docker Compose v2 ; sur la v1 il fallait contourner le problème par une boucle d'attente.

Le montage en `:ro` (lecture seule) pour le service `serve` renforce la sécurité : le conteneur d'inférence ne peut pas altérer le modèle.

Limite à connaître sur les permissions du volume

Le conteneur `train` s'exécute en `root` et écrit donc `model.joblib` avec l'UID 0. Le service `serve` s'exécutant sous `appuser` (UID 10001) peut le lire— les fichiers créés par `joblib.dump` ont les permissions `644`— mais ne pourrait pas l'écrire. C'est souhaitable ici. Si le volume devait être modifiable, il faudrait soit aligner l'UID du service d'entraînement sur 10001, soit appliquer un `chown` explicite à l'entrée.

4.2 Analyse de vulnérabilités avec Trivy

4.2.1 Résultats globaux

Le rapport `trivy.checks.txt` a été obtenu sur l'image finale `adult-serve:opt`, basée sur `debian 13.7`. Il dénombre **192 vulnérabilités** au total, réparties en deux familles :

Famille	Total	Critical	High	Medium	Low
Debian (paquets système)	167	0	45	59	61
Python (paquets pip)	25	0	7	15	3
Total	192	0	52	74	64

TABLE 7 – Vulnérabilités détectées par Trivy sur l'image finale, par famille.

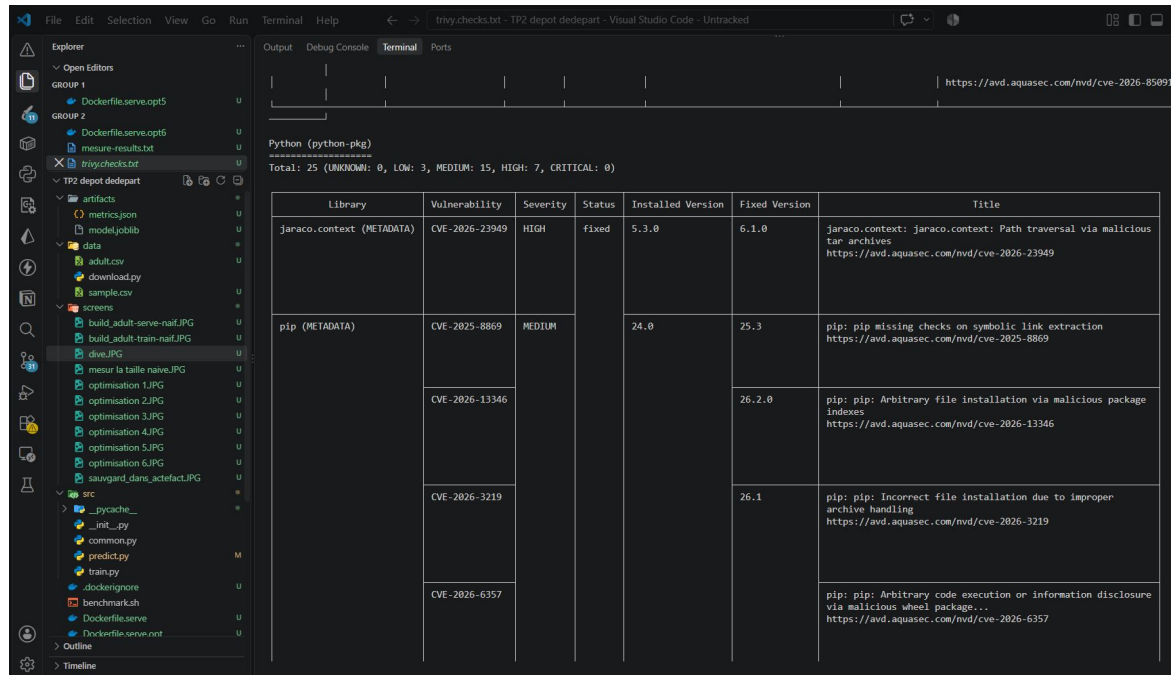


FIGURE 13 – Rapport Trivy

4.2.2 Analyse des vulnérabilités Python

Les 25 vulnérabilités Python se décomposent en quatre paquets principaux :

Paquet	Version	CVEs	Sév. max	Version corrigée
starlette	0.41.3	7	HIGH	1.3.1
pip	24.0	6	MEDIUM	26.2.0
setuptools	79.0.1	1	MEDIUM	83.0.0
wheel	0.45.1	1	HIGH	0.46.2
jaraco.context	5.3.0	1	HIGH	6.1.0
Total	—	16		

TABLE 8 – Vulnérabilités Python par paquet et version corrigée.

Cause racine identifiée : le *pinning* de FastAPI

La vulnérabilité la plus grave du côté Python, CVE-2025-62727 (*Starlette DoS via Range header merging*, sévérité HIGH), est **structurellement incorrigible** dans la configuration actuelle. En effet, `requirements.txt` épingle `fastapi==0.115.5`, dont la métadonnée impose `starlette>=0.40.0,<0.42.0`. Or la version de Starlette qui corrige la CVE est 0.49.1. Il est donc *mathématiquement impossible* de corriger cette CVE sans modifier la contrainte sur FastAPI.

La correction exige de passer à `fastapi>=0.121` (ou une version récente, 0.142.2 à la date du rapport), qui autorise `starlette>=0.46.0`, puis de laisser le solveur choisir la version corrigée de Starlette.

4.2.3 Vulnérabilités Debian

Les 167 vulnérabilités Debian se concentrent sur des paquets système ne contenant aucun code métier :

Paquet	CVEs	Nature de la vulnérabilité
<code>glibc</code>	21	débordements de tampon, DoS, élévation de privilèges
<code>util-linux</code>	6	privilege escalation via <code>X-mount</code>
<code>systemd</code>	7	élévation de privilèges locale (<code>systemd-homed</code>)
<code>tar</code>	5	TOCTOU, injection de fichiers cachés
<code>perl</code>	5	exécution de code, DoS
<code>sqlite3</code>	4	DoS, divulgation d'information
<code>coreutils</code>	4	courses, DoS
<code>ncurses</code>	2	débordement de tampon
<code>libpcre2</code>	1	écriture hors limites

TABLE 9 – Principaux paquets Debian affectés et nature des vulnérabilités.

4.2.4 Duplication des alertes : un signal trompeur

Un point d'analyse méthodologique mérite d'être souligné, car il change complètement la lecture du chiffre de 167. Une même CVE de `glibc` apparaît dans le rapport à la fois pour `libc-bin` et `libc6`, qui sont deux paquets binaires du même paquet source. De même, les 6 CVE `util-linux` sont comptées pour 9 paquets (`util-linux`, `mount`, `libmount1`, `libblkid1`, `libsmartcols1`, `libuuid1`, `liblastlog2-2`, `login`, `bsdutils`). Un décompte manuel des CVE *distinctes* de la section Debian donne environ **72 à 73 identifiants uniques**, contre 167 signalements.

Lecture correcte du rapport Trivy

Les 167 alertes Debian ne correspondent pas à 167 failles distinctes, mais à environ 73 CVE distinctes répliquées sur les paquets binaires d'un même paquet source. La charge de remédiation réelle est donc nettement inférieure à ce que suggère le décompte brut, et un rapport Trivy doit toujours être désagrégé avant d'être priorisé.

4.2.5 Plan de remédiation

Quatre actions, par ordre de rapport impact/effort :

1. **Mettre à jour la base système**— traite la quasi-totalité des 167 alertes Debian. À insérer dans le `Dockerfile.serve.opt` juste après le FROM de l'image finale :

```
RUN apt-get update \
    && apt-get upgrade -y \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

Le `rm -rf /var/lib/apt/lists/*` est indispensable : sans lui, les listes de paquets (plusieurs dizaines de Mo) restent dans la couche.

2. **Supprimer les paquets de construction de l'image finale** — le `pip`, `setuptools` et `wheel` embarqués dans `/opt/venv` sont inutiles à l'exécution. Leur suppression élimine 9 des 25 alertes Python et réduit la taille de l'image :

```
RUN /opt/venv/bin/pip uninstall -y pip setuptools wheel
```

3. **Mettre à jour `requirements.txt`** — correction de la cause racine identifiée en §4.3 :

```
fastapi>=0.121                # au lieu de ==0.115.5
uvicorn[standard]>=0.32.1
pydantic>=2.10.3
starlette>=1.3.1             # corrige CVE-2025-62727 (HIGH)
```

4. **Adopter une base sans gestionnaire de paquets** (distroless) — solution la plus efficace pour l'axe II : la suppression `glibc`, `systemd`, `perl`, `tar` et `sqlite3` de l'image finale, ramenant le compte de vulnérabilités à quelques unités. Cette orientation prépare directement le TP 3.

Effet attendu de la remédiation

Indicateur	Avant	Après	Reduction
Taille de l'image	176 Mo	130–140 Mo	~ -20 %
Vulnérabilités Debian	167	0	-100 %
Vulnérabilités Python	25	0	-100 %
Vulnérabilités HIGH	52	0	-100 %

Où sont les captures ?

Les treize captures du dossier `screens/` ne sont pas regroupées en annexe : chacune est insérée dans le corps du rapport, à l'emplacement exact de l'étape qu'elle documente. Ainsi 5 à 10 accompagnent chacune des six optimisations du chapitre 3, 12 accompagne la question 3 et 13 les résultats Trivy. La correspondance complète, qui permet de remonter d'une mesure à sa capture, est la suivante :

- figures 1 à 4 : version naïve ;
- figures 5 à 10 : les six optimisations, dans l'ordre ;
- figure 11 : synthèse des huit mesures ;
- figure 12 : question 3 ;
- figure 13 : résultats Trivy.

Chapitre 5

Conclusion

Ce TP avait pour objectif de conteneuriser un modèle d'intelligence artificielle et d'optimiser son image de manière mesurée. Les résultats obtenus sont convergents.

Sur le plan de la taille, l'image est passée de **624 Mo** à **176 Mo**, soit une réduction de **71,8 %**, obtenue pour l'essentiel par deux optimisations indépendantes : le choix d'une base **slim** (**-361 Mo**, 80,6 % du gain total) et l'ajout de **--no-cache-dir** (**-91 Mo**, soit 51,7 % de la taille de l'image finale). Cette dernière valeur est l'illustration la plus parlante du gaspillage classique en conteneurs : le cache du gestionnaire de paquets, figé dans une couche, représenterait à lui seul plus de la moitié de l'image finale.

Sur le plan de l'itération, le temps de reconstruction après modification du code est passé de **106 s** à **14 s**, soit une division par **7,6**. Ce gain, entièrement dû à l'ordonnancement des instructions **COPY**, est autrement plus précieux que le gain de taille en pratique quotidienne : c'est lui qui rend la boucle "modifier — rebuild — tester" utilisable. Cette différence de mécanisme— la taille dépend du *contenu* de l'image, le temps de reconstruction de la *topologie* du graphe de couches— constitue la réponse à la question 2 du support et la leçon principale du TP.

L'image finale est par ailleurs prête pour Kubernetes : utilisateur non-root, sondes **/health** et **/ready** distinctes, **ENTRYPOINT** en forme exec et **EXPOSE**. Le passage de l'optimisation 6 correspond exactement aux **securityContext** et aux sondes du TP 3.

L'analyse Trivy a mis en évidence un résultat contre-intuitif qui mérite d'être retenu. Sur les 192 vulnérabilités détectées, la grande majorité sont des *doublons* : une même CVE de **glibc** apparaît sous **libc-bin** et **libc6**, les 6 CVE **util-linux** sous 9 paquets. Le décompte brut de 167 alertes Debian ne correspond qu'à environ 73 CVE distinctes. Par ailleurs, la vulnérabilité Python la plus grave est structurellement incorrigible en l'état : l'épingleage **fastapi==0.115.5** interdit implicitement **starlette<0.42.0**, alors que la correction de CVE-2025-62727 exige **starlette>=0.49.1**. Identifier une *cause racine* dans les métadonnées de dépendances plutôt que de se contenter d'une mise à jour aveugle est la compétence mise en œuvre ici.

Trois écarts principaux subsistent et constituent la suite naturelle du travail : l'orchestration Compose n'est pas implémentée alors qu'elle fait partie de l'Axe II ; aucune remédiation n'a été appliquée aux vulnérabilités identifiées ; et la mesure du temps de construction de l'optimisation 3 (**678 s**) doit être rejouée, cette valeur étant cohérente avec aucune autre mesure du tableau. Le passage à une base **distroless**, combiné à la mise à jour système et à la correction du *pinning* FastAPI, permettrait de ramener le compte de vulnérabilités à quelques unités.